Région Hauts-de-France

Cofinancé par l'Union européenne

realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY 6 – La sécurité User et Hashage de mot de passe

Introduction

Dans ce chapitre nous allons voir la sécurité et le compte utilisateur

I) Le bundle Security

Allons voir la documentation de symfony . La sécurité est gérée par le bundle security de symfony.

Il est possible de l'installer via la commande que nous avons dans la documentation sauf que pour nous lors de l'installation il a été installé d'office

De cette installation en découle un fichier security.yaml dans le dossier config.

Allons dans notre dossier config puis le dossier package et allons chercher le fichier securtity.yaml

Et regardons le de plus près :

```
config > packages > security.yaml > {} security
You, 6 days ago | 1 author (You)
security:        You, 6 days ago • first commit …
    # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
    providers:
        users_in_memory: { memory: null }
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: users_in_memory

            # activate different ways to authenticate
            # https://symfony.com/doc/current/security.html#the-firewall

            # https://symfony.com/doc/current/security/impersonating_user.html
            # switch_user: true

    # Easy way to control access for large sections of your site
    # Note: Only the *first* access control that matches will be used
    access_control:
        # - { path: ^/admin, roles: ROLE_ADMIN }
        # - { path: ^/profile, roles: ROLE_USER }
```

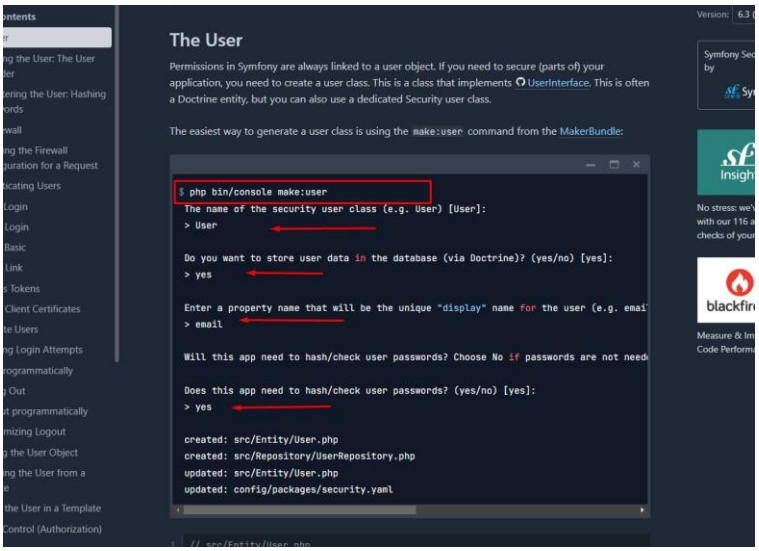
```
when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take time. This is
            # important to generate secure password hashes. In tests however, secure hashes
            # are not important, waste resources and increase test times. The following
            # reduces the work factor to the lowest possible values.
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
                algorithm: auto
                cost: 4 # Lowest possible value for bcrypt
                time_cost: 3 # Lowest possible value for argon
                memory_cost: 10 # Lowest possible value for argon
```

Le security se présente comme ci-dessus regardons plus dans les détails et comprenons le :

- Le password hasher : pour hasher les mots de passe (les enconder)
- Les providers : va être le fournisseur d'utilisateur
- Les firewall : parefeu, cœur de la sécurisation, chaque requête va passer par les firewalls, on va en définir plusieurs de base nous avons le dev et le main
- Access Control : le vérificateur avec le système de rôle

II) Le User

Si nous continuons sur la documentation on nous dit comment créer un User :



Créons donc notre User pas à pas

- Faisons la commande « php bin/console make:user »
- Stockons nos user dans notre bdd on répond yes
- Choisir le type de l'identifiant de l'utilisateur nous voulons l'email
- Et nous répondons que oui nous voulons hasher les mots de passe de l'utilisateur (une obligation RGPD)

```
created: src/Entity/User.php
created: src/Repository/UserRepository.php
updated: src/Entity/User.php
updated: config/packages/security.yaml

 Success!

Next Steps:
  - Review your new App\Entity\User class.
  - Use make:entity to add more fields to your User entity and then run make:migration.
  - Create a way to authenticate! See https://symfony.com/doc/current/security.html
```

Bien sur nous le User est une entity il faut donc faire une migration comme précisé ici

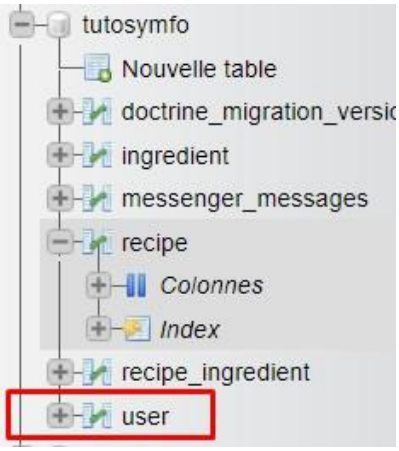
- « Php bin/console make:migration »

```
PS C:\Users\alexa\OneDrive\Bureau\workspace\symfonytuto\firstproject> php bin/console make:migrati

 [OK] Success!

 Next: Review the new migration "migrations/Version20230604072047.php"
 Then: Run the migration with php bin/console doctrine:migrations:migrate
 See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html
PS C:\Users\alexa\OneDrive\Bureau\workspace\symfonytuto\firstproject>
```

- Puis nous allons l'envoyer en bdd toujours de le même façon qu'une entity puisque user est une entity « php bin/console doctrine:migrations:migrate »
- Regardons dans notre Bdd si notre user s'est bien mis en place



Nous voyons que différents attributs ont été créés comme :

- Id
- Email
- Roles
- Password

Les Roles vont nous permettre d'intensifier la sécurité

Pour le reste vous connaissez

Regardons dans l'entity User dans symfony pour voir ce qui est utilisé

```php
<?php

namespace App\Entity;

use App\Repository\UserRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;

#[ORM\Entity(repositoryClass: UserRepository::class)]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180, unique: true)]
```

Nous voyons que le User utilise Le UserInterface et le PasswordAuthenticateUserInterface

Ce sont des interfaces qui vont permettre à symony de gérer correctement la sécurité

Maintenant ajoutons des propriétés dans l'entity User comme ci-dessous

- Faisons la commande « php bin/console make:entity »
- Nous allons dire que nous voulons l'entity User
- Nous allons ajouter un name qui est un string avec une longueur de 50 not null
- Nous voulons un pseudo qui sera un string d'une longueur de 50 qui peut être null
- createdAt pour le type un dateTimeImmutable non null
- et c'est tout

Faisons à nouveau notre migration et envoyons la migration en bdd A vous de vérifier si tout est bien mis en bdd

Structure de table | Vue relationnelle

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	id	int			Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
2	email	varchar(180)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
3	roles	json			Non	Aucun(e)			Modifier Supprimer Plus
4	password	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
5	name	varchar(50)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
6	pseudo	varchar(50)	utf8mb4_unicode_ci		Oui	NULL			Modifier Supprimer Plus
7	created_at	datetime			Non	Aucun(e)	(DC2Type:datetime_immutable)		Modifier Supprimer Plus

Tout cocher Avec la sélection : Parcourir Modifier Supprimer Primaire Unique Index Spatial Texte entier

Nous allons continuer à travailler dans l'entity et nous allons créer dans l'entity User un private $plainPassword

```
18      #[ORM\Column(length: 180, unique: true)]
19      private ?string $email = null;
20
21      #[ORM\Column]
22      private array $roles = [];
23
24      private ?string $plainPassword = null;
25
26      /**
27       * @var string The hashed password
```

Nous verrons par la suite ce que nous allons en faire …

Créons le getter et le setter du $plainPassword

```php
    /**
     * Get the value of plainPassword
     */
    public function getPlainPassword()
    {
        return $this->plainPassword;
    }

    /**
     * Set the value of plainPassword
     *
     * @return  self
     */
    public function setPlainPassword($plainPassword)
    {
        $this->plainPassword = $plainPassword;

        return $this;
    }
```

Il faut également créer dans l'entity le construcs comme fait précédemment dans les autre entity

```php
    #[ORM\Column]
    private ?\DateTimeImmutable $createdAt = null;

    public function __construct()
    {
        $this->createdAt = new \DateTimeImmutable();
    }

    public function getId(): ?int
    {
        return $this->id;
```

Quand la date est instancié on veut que ce soit un objet DateTimeImmutable

Place maintenant à la validation avec les constraints

- les uses

```php
use App\Repository\UserRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;
use Symfony\Component\Validator\Constraints as Assert;
use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;
```

- utilisation de l'uniqueEntity par rapport à l'email

```
9   use Symfony\Component\Validator\Constraints as Assert;
10  use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;
11
12  #[UniqueEntity('email')]
13  #[ORM\Entity(repositoryClass: UserRepository::class)]
14  class User implements UserInterface, PasswordAuthenticatedUserInterfac
15  {
16      #[ORM\Id]
```

- utilisation du NotBlank et du Length pour le name

```
33      private ?string $password = null;
34
35      #[Assert\NotBlank()]
36      #[Assert\Length(min:2,max:50)]
37      #[ORM\Column(length: 50)]
38      private ?string $name = null;
39
40      #[ORM\Column(length: 50, nullable: true)]
41      private ?string $pseudo = null;
```

- Utilisation du Length pour le pseudo

```
38      private ?string $name = null;
39
40      #[Assert\Length(min:2,max:50)]
41      #[ORM\Column(length: 50, nullable: true)]
42      private ?string $pseudo = null;
43
44      #[ORM\Column]
45      private ?\DateTimeImmutable $createdAt = null;
```

- Utilisation de Length également pour le mail et aussi du Assert\email

```
17      #[ORM\GeneratedValue]
18      #[ORM\Column]
19      private ?int $id = null;
20
21      #[Assert\Length(min:2, max:180)]
22      #[Assert\Email()]
23      #[ORM\Column(length: 180, unique: true)]
24      private ?string $email = null;
25
26      #[ORM\Column]
27      private array $roles = [];
```

-

- Le NotNull pour les roles

```
23      #[ORM\Column(length: 180, unique: true)]
24      private ?string $email = null;
25
26      #[ORM\Column]
27      #[Assert\NotNull()]
28      private array $roles = [];
29
```

- Le NotBlank sur le password

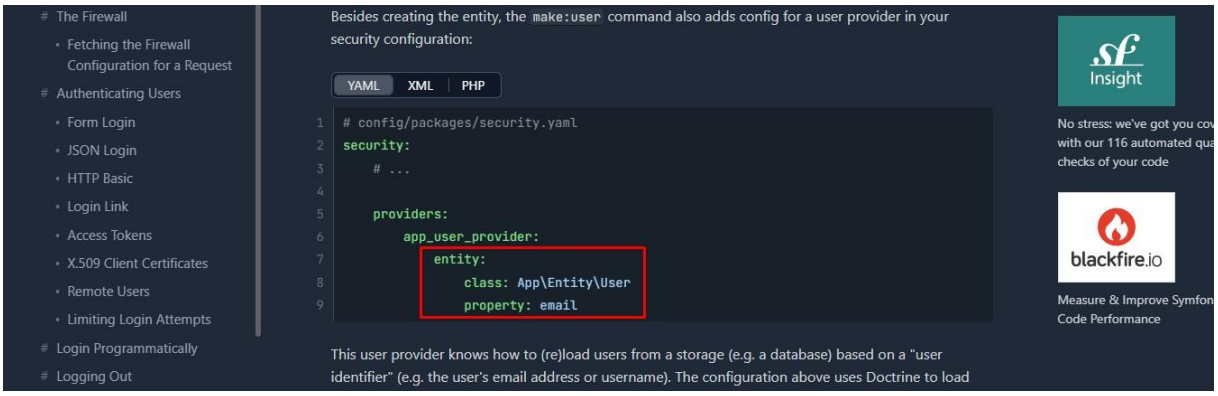
```
32      /**
33       * @var string The hashed password
34       */
35      #[ORM\Column]
36      #[Assert\NotBlank()]
37      private ?string $password = null;
38
39      #[Assert\NotBlank()]
40      #[Assert\Length(min:2,max:50)]
41      #[ORM\Column(length: 50)]
```

- Le NotNull sur le createdAt

```
45      #[ORM\Column(length: 50, nullable: true)]
46      private ?string $pseudo = null;
47
48      #[ORM\Column]
49      #[Assert\NotNull()]
50      private ?\DateTimeImmutable $createdAt = null;
51
```

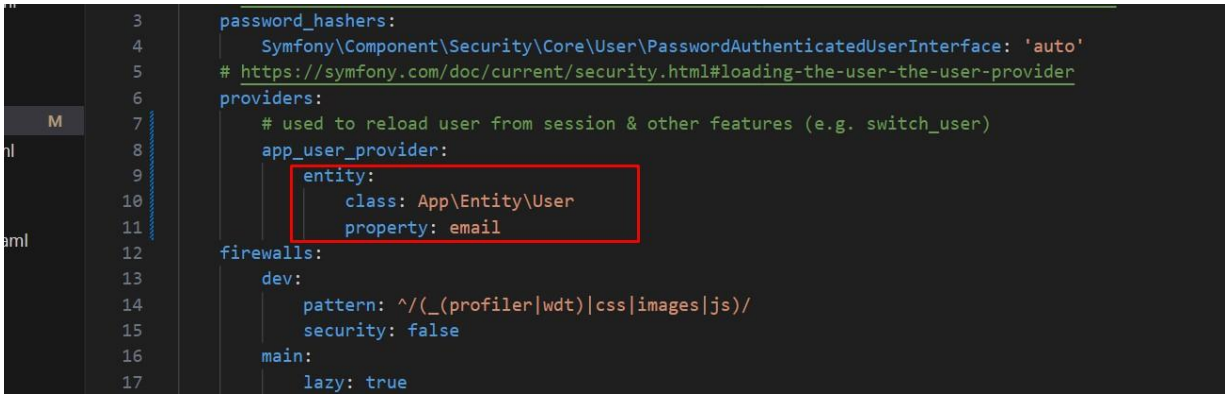
On ne met pas de décorateur column ou autre sur le plainPassword pour ne pas stocker le mdp en clair dans la bdd.

Suivons la documentation et il est spécifier de faire le loading du user.



Le provider va nous permettre de récupérer un utilisateur de sa zone de stockage

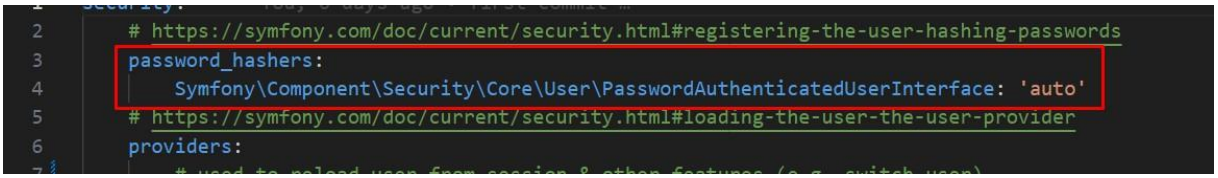
Nous nous apercevons que ce bout de code est déjà présent dans le security.yaml



```
3    password_hashers:
4        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
5    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
6    providers:
7        # used to reload user from session & other features (e.g. switch_user)
8        app_user_provider:
9            entity:
10               class: App\Entity\User
11               property: email
12   firewalls:
13       dev:
14           pattern: ^/(_(profiler|wdt)|css|images|js)/
15           security: false
16       main:
17           lazy: true
```

III) Le hash du mot de passe

Nous pouvons voir dans notre security.yaml que le hash du password est créé automatiquement



```
2    # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
3    password_hashers:
4        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
5    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
6    providers:
```

A noter que Symfony prend le hasher le plus sure et qu'il est mis à jour au fur et à mesure du temps.

IV) Les fixtures utilisateurs

Nous allons créer des users dans nos fixtures. Allons dans notre AppFixtures

```php
        $manager->persist($recipe);

    }

    //Users
    for ($i=0; $i <10 ; $i++) {
        $user = new User();
        $user -> setName($this->faker->name())
            ->setPseudo(mt_rand(0,1)===1 ? $this->faker->firstName():null)
            ->setEmail($this->faker->email())
            ->setPassword('password');

        $manager->persist($user);      You, now • Uncommitted changes
    }

    $manager->flush();
```

Essayons l'envoie des données en bdd « php bin/console doctrine:fixtures:load » et regardons notre table user dans notre bdd

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

←T→	id	email	roles	password	name	pseudo	created_at (DC2Type:datetime_immutable)
Éditer Copier Supprimer	1	alex.begue@blin.org	[]	password	Nathalie Poulain	Grégoire	2023-06-04 09:11:07
Éditer Copier Supprimer	2	catherine15@charpentier.fr	[]	password	Alphonse Caron	NULL	2023-06-04 09:11:07
Éditer Copier Supprimer	3	psanchez@live.com	[]	password	Benjamin De Oliveira	Thierry	2023-06-04 09:11:07
Éditer Copier Supprimer	4	marie40@pages.com	[]	password	Marianne Leroy-Pottier	NULL	2023-06-04 09:11:07
Éditer Copier Supprimer	5	klein.marcelle@david.com	[]	password	Nathalie Auger-Denis	NULL	2023-06-04 09:11:07
Éditer Copier Supprimer	6	meyer.victor@club-internet.fr	[]	password	Nicole Barre-Charles	Sébastien	2023-06-04 09:11:07
Éditer Copier Supprimer	7	adele58@julien.com	[]	password	Frédéric Gros	Célina	2023-06-04 09:11:07
Éditer Copier Supprimer	8	laurent30@orange.fr	[]	password	Henri de Buisson	Sophie	2023-06-04 09:11:07
Éditer Copier Supprimer	9	vincent.marques@maillot.com	[]	password	René Merle	NULL	2023-06-04 09:11:07
Éditer Copier Supprimer	10	obonnet@baudry.net	[]	password	Noël Gay-Chartier	Éric	2023-06-04 09:11:07

Tout cocher | Avec la sélection : Éditer Copier Supprimer Exporter

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Pour le moment nous avons un problème avec les roles et le password

- Pour les roles nous avons oublier de faire notre setRoles dans le AppFixtures faisons le de suite ?

```php
    //Users
    for ($i=0; $i <10 ; $i++) {
        $user = new User();
        $user -> setName($this->faker->name())
            ->setPseudo(mt_rand(0,1)===1 ? $this->faker->firstName():null)
            ->setEmail($this->faker->email())
            ->setRoles(['ROLE_USER'])      You, 1 second ago • Uncommitted changes
            ->setPassword('password');

        $manager->persist($user);
    }

    $manager->flush();
}
```

- Envoyons de nouveau en bdd nos fausses données et regardons ce qu'il se passe

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

	id	email	roles	password	name	pseudo	created_at (DC2Type:datetime_immutable)
Éditer Copier Supprimer	11	jacques.marcelle@peltier.com	["ROLE_USER"]	password	Nicolas Morel-Fernandez	Matthieu	2023-06-04 09:16:55
Éditer Copier Supprimer	12	dominique06@prevost.fr	["ROLE_USER"]	password	Céline-Hélène Clerc	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	13	richard.henri@wanadoo.fr	["ROLE_USER"]	password	Georges Dubois	Adèle	2023-06-04 09:16:55
Éditer Copier Supprimer	14	margaux.reynaud@morvan.org	["ROLE_USER"]	password	Arnaude Legrand	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	15	bernier.hortense@hotmail.fr	["ROLE_USER"]	password	Augustin Normand	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	16	wdupuy@duval.fr	["ROLE_USER"]	password	Richard Joseph-Alves	Camille	2023-06-04 09:16:55
Éditer Copier Supprimer	17	olamy@cohen.com	["ROLE_USER"]	password	Eugène Picard	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	18	dumont.david@bouvet.org	["ROLE_USER"]	password	Noël Remy-Sauvage	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	19	clement.charlotte@free.fr	["ROLE_USER"]	password	Tristan Martel	NULL	2023-06-04 09:16:55
Éditer Copier Supprimer	20	rrousset@grondin.net	["ROLE_USER"]	password	Thérèse de la David	NULL	2023-06-04 09:16:55

Tout cocher | Avec la sélection : Éditer | Copier | Supprimer | Exporter

Nous voyons que les roles sont bien mis en place

Occupons nous maintenant du password qui n'est pas encodé pour cela regardons la doc et nous allons devoir appeler le UserPasswordHasherInterface pour hasher le mot de passe

```php
// src/Controller/RegistrationController.php
namespace App\Controller;

// ...
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class RegistrationController extends AbstractController
{
    public function index(UserPasswordHasherInterface $passwordHasher): Response
    {
        // ... e.g. get the user data from a registration form
        $user = new User(...);
        $plaintextPassword = ...;

        // hash the password (based on the security.yaml config for the $user class)
        $hashedPassword = $passwordHasher->hashPassword(
            $user,
            $plaintextPassword
        );
```

Faisons de même donc dans notre AppFixture

- Le use

```php
use Doctrine\Bundle\FixturesBundle\Fixture;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;
```

- Au niveau du construct

```php
    private UserPasswordHasherInterface $hasher;

    public function __construct(UserPasswordHasherInterface $hasher)
    {
        $this->faker = Factory::create('fr_FR');
        $this->hasher = $hasher;
    }
```

- Dans notre code d'envoie de fixtures

```php
        //Users
        for ($i=0; $i <10 ; $i++) {
            $user = new User();
            $user -> setName($this->faker->name())
                ->setPseudo(mt_rand(0,1)===1 ? $this->faker->firstName():null)
                ->setEmail($this->faker->email())
                ->setRoles(['ROLE_USER']);

            $hashPassword = $this->hasher->hashPassword(
                $user,
                'password'
            );
            $user->setPassword($hashPassword);        You, 6 seconds ago • Uncommit


            $manager->persist($user);
        }

        $manager->flush();
    }
```

Envoyons nos données en bdd et regardons ce qu'il s'y passe

	id	email	roles	password	name	pseudo	created_at (DC2Type:datetime_immutable)
Éditer Copier Supprimer	31	nmary@charpentier.fr	["ROLE_USER"]	$2y$13$r0rrVRIwjNIWdamrx2gBs.m2356ET8fdTk8z/sExxXj...	Stéphanie-Célina Imbert	Laure	2023-06-04 09:30:19
Éditer Copier Supprimer	32	anastasie.voisin@gimenez.net	["ROLE_USER"]	$2y$13$Zle4DKI4z6QiTV9KleeR0eC.IRqvswFII9inYHLXWJu...	Sébastien Coulon	Laure	2023-06-04 09:30:19
Éditer Copier Supprimer	33	victoire72@noos.fr	["ROLE_USER"]	$2y$13$XwrhoVQVYC2arMp5AcBfeuXQ3FtWKQH78N/eHMADeQ4...	Jean Millet	Tristan	2023-06-04 09:30:20
Éditer Copier Supprimer	34	corinne20@berger.com	["ROLE_USER"]	$2y$13$hxIwAKpY2LJYWRSgR/Yyk.8CIWV4/kOVR7wmM38173q...	Théodore Maillard	Adèle	2023-06-04 09:30:20
Éditer Copier Supprimer	35	zacharie.baron@live.com	["ROLE_USER"]	$2y$13$L2q7VbGAHnMM.Qdqzdw6P.mN1IhficXh5PYz36CyDMh...	Chantal Legrand	NULL	2023-06-04 09:30:21
Éditer Copier Supprimer	36	picard.alfred@live.com	["ROLE_USER"]	$2y$13$57S4U4fjIujBg2c10HxqhOERueZmIPcs1S4yM3nYI/V...	Susanne Bigot	Émile	2023-06-04 09:30:21
Éditer Copier Supprimer	37	margot31@gmail.com	["ROLE_USER"]	$2y$13$ypL2GW1iwaIqYQpbqb0aauwTgu8HNDXdgnoFvvr85nZ...	Franck Jean-Cordier	NULL	2023-06-04 09:30:21
Éditer Copier Supprimer	38	lucy64@club-internet.fr	["ROLE_USER"]	$2y$13$09ZkFBJC5j/Pf1sjRvJ6HuSh2nPQbXjx7OipQ51amYu...	Marthe Michel	NULL	2023-06-04 09:30:22
Éditer Copier Supprimer	39	william.chartier@imbert.fr	["ROLE_USER"]	$2y$13$n/yZWMbXrcgtbKv2Gm8NnObyI3y50aXIJTIj4kn8EG1...	Augustin Buisson	Thibaut	2023-06-04 09:30:22
Éditer Copier Supprimer	40	herve.henriette@benard.fr	["ROLE_USER"]	$2y$13$x/6uPKirufPZaMIMNhMXe.WWWxBRjyCGc6m4s51XVRe...	Henriette Mace	Charles	2023-06-04 09:30:23

Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

Nos mots de passe sont bien hashés, cela fonctionne très bien

Dans le prochain chapitre nous verrons comment nous pouvons faire autrement et plus simplement avec les Entity Listener